Midterm Study Sheet

EE1021 Programming for Engineers — what the exam covers (weeks 1–6), how the questions look, and practice with answers

Basri Erdoğan — 2026-2027 Fall

Rule: try every practice item on paper before you open the answers at the end. The exam is written by hand, closed book, 80 minutes. Practice the same way: no computer, no VS Code, no phone, a pen and this sheet. If an answer box is too small, continue on the back of the same sheet and write *see back* in the box.

When and where: Wednesday **4 November 2026, 10:00** (not 09:00), 80 minutes. Two rooms: Ataköy 4th floor C corridor, rooms 16-18, and 3rd floor C corridor, rooms 3-5. Which students sit in which room will be announced on CATS. Bring your student ID card and a pen.

How the exam works

Section	Questions	Points	What you do
A — Predict the output	6 × 5p	30	Read a short program. Write **exactly** what it prints, one printed line per line. If a line stops the program, write *Error* and the error type.
B — Multiple choice	4 × 5p	20	One correct answer per question.
C — Fill in the blank	2 × 5p	10	Q11: a program with blanks; the comments show what it must print. Q12: a short program and its traceback; fill in the error type, the value that caused it, the failing line and its function, the line that called that function, and the kind of error.
D — Short code	4 questions (8 + 10 + 10 + 12p)	40	Write three functions and one small class. You get the header, an example call and its output.

The exam covers **weeks 1–6**: L01 (values, strings, operators, `input`/`print`), L02 (`if`, `while`, `for`, `range`, `break`), L03 (functions, parameters, `return`, scope, modules), L04 (tuples, lists, comprehensions, aliasing), week 5 (dictionaries; errors, `try`/`except`, `raise`) and L07 (classes, objects, methods, `__init__`, `__str__`). **Not** in the exam: sets, `dict.get`, `try … else`, NumPy, files.

There is **no reference box**: you need to know the built-in functions (`print`, `input`, `int`, `float`, `str`, `len`, `sum`, `min`, `max`, `sorted`, `range`, `type`), the list methods (`append`, `extend`, `insert`, `remove`, `pop`, `copy`, `sort`) and the dictionary operations (`d[key]`, `key in d`, `del`, `pop`, `items`, `values`) by name.

How to write a Section A answer

Python prints	Example	Printed
`print(a, b)` puts **one space** between items	`print("x", 3)`	`x 3`
each `print` starts a new line	two `print` calls	two lines
`/` always gives a `float`	`print(6 / 3)`	`2.0`
`//` and `%` on `int` give `int`	`print(17 // 5, 17 % 5)`	`3 2`
a list: brackets, comma + space	`print([1, 2])`	`[1, 2]`
a string inside a list or dictionary: single quotes	`print(["a"])`	`['a']`
a dictionary	`print({"S1": 4})`	`{'S1': 4}`
a Boolean	`print(3 > 2)`	`True`
a function without `return` gives `None`	`print(print("hi"))`	`hi`, then `None`

Python prints	Example	Printed
an object with `__str__`	`print(obj)`	whatever `__str__` **returns**

How to trace a loop on paper: make a table with one row per repetition and one column per variable. Write each value **after** the line that changes it. Stop as soon as the condition is false or a `break` runs. Then write what is printed.

Week 1 — Values, strings, operators, input and output

Rule	Example
Indexing starts at 0; `-1` is the last character	`"robot"[0]` is `"r"`, `"robot"[-2]` is `"o"`
A slice `[start:stop:step]` **excludes** `stop`	`"abcdefgh"[3:6]` is `"def"`, `[::2]` is `"aceg"`, `[::-1]` reverses
An index past the end is an error, a slice is not	`"abc"[3]` → `IndexError`; `"abc"[1:10]` is `"bc"`
`in` on strings tests a substring	`"bot" in "robotics"` is `True`
`+` joins strings (no space added), `*` repeats	`"ab" + "c"` is `"abc"`; `"ab" * 3` is `"ababab"`
`input()` **always** returns a string	user types 7: `input() * 2` is `"77"`; `int(input()) * 2` is `14`
`int("4.5")` and `float("abc")` raise `ValueError`	use `float()` for text with a decimal point
`=` binds a name, `==` compares	`x == 5` is an expression with a `bool` value
Precedence: `**`, then `* / // %`, then `+ -`	`4 + 2 * 3` is `10`; `(4 + 2) * 3` is `18`
Names: letters, digits, `_`; not starting with a digit; case-sensitive	`total_1` valid; `1total`, `my-name` invalid

Week 2 — Decisions and loops

Rule	Example
`if / elif / else` runs **the first** true branch only	order the tests from the special case down
Indentation is the block	a line aligned with `if` runs after the whole `if`
`range(start, stop, step)` excludes `stop`	`range(2, 11, 2)` → 2 4 6 8 10; `range(5, 0, -1)` → 5 4 3 2 1
A positive step with `start > stop` gives nothing	`range(10, 1, 2)` is empty
A `while` loop needs a start value, a condition and an update	no update → infinite loop
`break` leaves the innermost loop **immediately**	lines after `break` in the body do not run
Accumulate / count / build	`total = 0` / `count = 0` / `text = ""` **before** the loop

Boundaries: test the exact limit and one step on each side. `x < 30` and `x <= 30` differ only at 30.

Week 3 — Functions and modules

Rule	Example
A definition stores the function; a **call** runs it	`def f(x): ...` prints nothing until `f(3)`
Arguments match parameters left to right	`area(5, 3)` → `length = 5`, `width = 3`
`return` sends a value back **and ends the call**	lines after the executed `return` do not run
`print` only displays; the call's value is then `None`	`y = show(5)` → `y` is `None`
A parameter is a **local** name	`x = x + 1` inside does not change the caller's `x`
A default argument is used when the call leaves it out	`def cal(v, off=0.05)`: `cal(4.9)` uses 0.05
Parameters with defaults come last	`def f(a, b=1)` valid; `def f(a=1, b)` is a `SyntaxError`
Modules: `import math` then `math.sqrt(2)`	your own `sensor_tools.py` → `import sensor_tools`

Week 4 — Tuples and lists

Rule	Example
The **comma** makes a tuple	`(4.92,)` is a tuple; `(4.92)` is a `float`
Tuples cannot be changed	`t[1] = 5` → `TypeError`
Unpacking needs the same number of names	`name, value = ("S1", 4.92)`
A function returns several values as one tuple	`return min(v), max(v)` → `low, high = f(v)`
`append(x)` adds **one** element; `extend(L)` adds **each** element	`[1].append([2, 3])` → `[1, [2, 3]]`; `extend` → `[1, 2, 3]`
`insert(i, x)` puts `x` at position `i`	`["a", "c"].insert(1, "b")` → `["a", "b", "c"]`
`remove(x)` deletes the **first** `x`; `pop(i)` deletes and **returns**	`pop()` without an index takes the last one
`sort()` changes the list and returns `None`; `sorted(L)` returns a new list	`L = L.sort()` is a bug: `L` becomes `None`
`b = a` is an **alias** (one list, two names)	`b.append(1)` also changes `a`
`a.copy()` or `a[:]` makes a **separate** list	changes to the copy do not reach `a`
A function that **mutates** a list parameter changes the caller's list	`values.append(5)` inside is visible outside
Rebinding a list parameter stays local	`values = [5]` inside does not change the caller
Comprehension: `[expression for x in seq if condition]`	`[v * 2 for v in [1, 5, 8] if v > 2]` → `[10, 16]`

Week 5 — Dictionaries, errors and exceptions

Rule	Example
A dictionary maps **keys** → **values**: `{"S1": 4.92}`	`d["S1"]` is `4.92`
`d[key] = v` adds a new key or replaces the value	a repeated key keeps the **last** value
A missing key in `d[key]` → `KeyError`	check first with `if key in d:`
`in` tests the **keys**	`4.92 in d` is `False`; `4.92 in d.values()` is `True`
`del d[key]` removes; `d.pop(key)` removes and returns	`d.pop("S9", None)` returns `None` for a missing key
`for k in d:` gives the keys; `for k, v in d.items():` gives the pairs	`d.values()` gives the values only
Keys must be immutable	strings, numbers, tuples; never a list

The counting pattern (learn it by heart):

```
counts = {}
for item in items:
    if item in counts:
        counts[item] += 1
    else:
        counts[item] = 1
```

Kind of error	When you see it	Example
Syntax error	before anything runs	missing `:` after `if`, wrong indentation
Runtime error (exception)	while a line runs; a traceback	`ValueError`, `KeyError`, `IndexError`, `ZeroDivisionError`, `TypeError`
Logic error	never: a wrong result and no message	`+ 32` written as `- 32`

- Read a traceback **from the last line up**: the error type, the failing line, then who called it.
- In `try`, Python stops at the line that raises; the rest of the `try` block is **skipped**, the first matching `except` runs, then the program continues **after** the whole statement.
- Keep the `try` block small, catch only the error you expect, never write a bare `except:`.
- `raise ValueError("message")` in a function reports input it cannot use; the caller decides what to do.

Week 6 — Classes and objects

Term	Meaning
class	a new type: `class Battery:`
object / instance	one value of that type: `pack = Battery(2000)`
`__init__`	the constructor; runs automatically when the object is created
`self`	the object the method was called on; you never pass it yourself
attribute	data stored on the object: `self.capacity = capacity`
method	a function inside the class: `pack.charge(500)` means `Battery.charge(pack, 500)`
command / query	a command changes state and returns nothing; a query returns information and changes nothing
`__str__`	must **return** a string; `print(obj)` uses it

- Every attribute must be stored on `self`. `x = x` inside `__init__` stores nothing on the object.
- Create a list attribute **inside** `__init__` (`self.readings = []`) so each object gets its own list.
- Two objects of one class have **separate** attributes: changing `a.level` does not change `b.level`.
- Validate in a method with `if` + `raise ValueError(...)` *before* storing the value.
- f-strings: `f"{self.name}: {self.ohms} ohm"` puts each value into the text.
- `assert condition, "message"` stops the program with `AssertionError` when the condition is false (tests).

Practice set

Work on paper. Cover the answers. Part A has the shape of Section A, Part B of Section B, C1 and C4 of Section C, Part D of Section D. C2 and C3 are extra practice for the counting pattern and for classes.

Part A — Predict the output

A1.

```
part = "capacitor"
k = 23
print(part[1:4], part[-2], len(part))
print(k // 4, k % 4, k / 2)
print("cit" in part, part[::3])
```

A2.

```
total = 0
n = 10
while n > 0:
    total += n
    n -= 3
    if total > 20:
        break
print(n, total)
```

A3. Careful with capital letters.

```
count = 0
for ch in "Engineering":
    if ch in "aeiou":
        count += 1
print(count)
```

A4.

```
def scale(x, factor=3):
    x = x * factor
    return x

def show(x):
    print("show:", x)

x = 4
y = scale(x)
z = scale(x, 0.5)
w = show(y)
print(x, y, z, w)
```

A5.

```
p = [5, 2, 9]
q = p
r = p[:]
q.insert(0, 7)
r.append(1)
s = sorted(r)
print(p)
print(r)
print(s)
print([v + 1 for v in p if v % 2 == 1])
```

A6.

```
grades = {"Ali": 70, "Ece": 85}
grades["Can"] = 60
grades["Ali"] = 75
removed = grades.pop("Ece")
print(grades)
print(removed, "Ece" in grades, 75 in grades)
for name, g in grades.items():
    print(name, g >= 70)
```

A7.

```
values = ["8", "zero", "0", "4"]
for text in values:
    try:
        n = int(text)
        print(12 // n)
    except ValueError:
        print("not a number")
    except ZeroDivisionError:
        print("divide by zero")
print("done")
```

A8.

```
class Counter:
    def __init__(self, start):
        self.value = start

    def step(self, amount=1):
        self.value += amount

    def __str__(self):
        return f"Counter at {self.value}"

c1 = Counter(0)
c2 = Counter(10)
c1.step()
c2.step(5)
c1.step(c2.value)
print(c1)
print(c2.value)
```

Part B — Concept questions (one answer each)

B1. What is `type(7 / 7)`? (a) `int` (b) `float` (c) `bool`

B2. Which values does `range(1, 10, 3)` give? (a) 1 4 7 (b) 1 4 7 10 (c) 3 6 9

B3. `a = [1, 2]` then `a.append([3, 4])`. What is `len(a)`? (a) 4 (b) 3 (c) 2

B4. `d = {"S1": 4.9}`. Which expression is `True`? (a) `4.9 in d` (b) `"S1" in d` (c) `"S1" in d.values()`

B5. `if x > 5 print(x)` is (a) a syntax error (b) a runtime error (c) a logic error.

B6. `print(int("4.5"))` raises (a) `TypeError` (b) `ValueError` (c) nothing; it prints `4`.

B7. `t = (1, 2)` then `t[0] = 5` raises (a) `TypeError` (b) `IndexError` (c) nothing.

B8. A `__str__` method contains `print(self.name)` and no `return`. What does `print(obj)` do? (a) prints the name (b) prints `None` (c) stops with `TypeError`

B9. `readings = []` is written directly under `class Sensor:`, outside `__init__`. What happens when two sensors `append` to `self.readings`? (a) each has its own list (b) both add to **one** shared list (c) `AttributeError`

Part C — Fill in the blank

C1. Make every `print` give the output in its comment.

```
tag = "TEMP-0425"
print(tag[ ____ : ____ ])                    # -> 0425
print(tag[ ____ ])                           # -> P   (use a negative index)

temps = [12, 25, 31, 8, 19]
warm = [ ________ for t in temps if ________ ]
print(warm)                                  # -> [250, 310, 190]
```

C2. Complete the counting loop.

```
colors = ["red", "blue", "red", "red", "green"]
counts = ____
for c in colors:
    if c ____ counts:
        counts[c] ____ 1
    else:
        counts[c] = ____
print(counts)        # -> {'red': 3, 'blue': 1, 'green': 1}
```

C3. Complete the class so that the program prints `L1: 60 W, on=True`.

```
class Lamp:
    def ________(self, name, watts):
        self.name = name
        self.watts = watts
        self.on = ________

    def switch_on(self):
        ____.on = True

    def __str__(self):
        ________ f"{self.name}: {self.watts} W, on={self.on}"

lamp = Lamp("L1", 60)
lamp.switch_on()
print(lamp)
```

C4. `report.py` (line numbers on the left) was run and printed the output below it. Fill in the blanks.

```
1  def to_volts(text):
2      return float(text)
3
4  def average(texts):
5      values = []
6      for t in texts:
7          values.append(to_volts(t))
8      return sum(values) / len(values)
9
10 print(average(["4.9", "5.1"]))
11 print(average(["5.0", "five", "4.8"]))
```

```
5.0
Traceback (most recent call last):
  File "report.py", line 11, in <module>
    print(average(["5.0", "five", "4.8"]))
  File "report.py", line 7, in average
    values.append(to_volts(t))
  File "report.py", line 2, in to_volts
    return float(text)
ValueError: could not convert string to float: 'five'
```

Question	Answer
(a) The error type	
(b) The value that caused the error	
(c) The line number where the error happened, and its function	
(d) The line number that called that function	
(e) Which kind of error is it: syntax, runtime or logic?	
(f) Extra: the line number in the main program where the failing call started	
(g) Extra: why was `5.0` printed before the traceback?	

Part D — Write the code

D1. Write `classify_temp(t)` that returns `"Freezing"` below 0, `"Cold"` from 0 up to but not including 15, `"Mild"` from 15 up to and including 25, and `"Hot"` above 25. Use `if / elif / else` and `return`. Check your answer with `-1, 0, 14, 15, 25, 26`.

D2. Write `even_stats(values)` that uses one `for` loop and returns a tuple `(count, total)` of the **even** numbers in the list. `even_stats([3, 4, 10, 7, 2])` returns `(3, 16)`.

D3. Write `count_letters(word)` that returns a dictionary with how often each character occurs. `count_letters("level")` returns `{'l': 2, 'e': 2, 'v': 1}`.

D4. Write `parse_numbers(texts)` that returns a tuple `(numbers, bad)`: `numbers` is the list of `float` values that convert, `bad` is how many texts raised `ValueError`. Use `try` / `except`. `parse_numbers(["2.5", "x", "4", ""])` returns `([2.5, 4.0], 2)`.

D5. A `Thermostat` has a name and a minimum and maximum temperature. Its target starts at the minimum. `set_target(t)` raises `ValueError` when `t` is below the minimum or above the maximum and otherwise stores it. It prints as `TH1: target 21 (16-28)`. Write the class, then a test that makes `TH1` with limits 16 and 28, sets 21, prints it, and catches the error of `set_target(30)` with `try` / `except` and prints `rejected`.

D6. Write `in_range(values, low, high)` that uses a `for` loop and `append` to return a **new** list with the values from `low` to `high` (both included), in their original order. The input list must not change. `in_range([4.1, 4.9, 5.3, 5.0], 4.75, 5.25)` returns `[4.9, 5.0]`.

D7. Write `largest(values)` that returns a tuple `(value, index)` with the largest value of a non-empty list and the position of its **first** occurrence. Do not use `max()`. `largest([3, 9, 2, 9])` returns `(9, 1)`.

D8. `prices` is a dictionary of part prices. Write `order_total(prices, items)` that returns a tuple `(total, unknown)`: the sum of the prices of the items in the list (an item can appear more than once), and how many items are not keys of `prices`. Use `in`, not `try`. `order_total({"R1": 2, "C1": 5}, ["R1", "X9", "C1", "R1"])` returns `(9, 1)`.

D9. An `Inventory` keeps a dictionary that maps part names to quantities. Write the class: `__init__(self)` starts with an empty dictionary; `add(self, part, n)` raises `ValueError` when `n` is not positive and otherwise increases the quantity (a new part starts at 0); `take(self, part, n)` raises `ValueError` when there are fewer than `n` of that part and otherwise decreases the quantity; `quantity(self, part)` returns the quantity, or `0` for a part that was never added; `__str__` returns `Inventory: 2 part types` (the number of keys).

Answers

Part A

A1.

```
apa o 9
5 3 11.5
True cat
```

`capacitor` has indices `c0 a1 p2 a3 c4 i5 t6 o7 r8`: `[1:4]` is 1, 2, 3; `[::3]` is 0, 3, 6. `23 / 2` is a `float` even though the division is not exact.

A2. `1 21`. Totals 10 → 17 → 21; `n` goes 10 → 7 → 4 → 1. After the third pass `total > 20`, so `break` leaves the loop before `n > 0` is tested again.

A3. 4. The vowels are `i, e, e, i`; the capital `E` is not in `"aeiou"`.

A4.

```
show: 12
4 12 2.0 None
```

`scale` changes its own `x`, not the caller's. `0.5` makes a `float`. `show` prints and returns `None`.

A5.

```
[7, 5, 2, 9]
[5, 2, 9, 1]
[1, 2, 5, 9]
[8, 6, 10]
```

`q` is an alias of `p`, so `insert` changes `p`. `r` is a copy made before the insert. `sorted` makes a new list. The comprehension reads `p` and keeps the odd values 7, 5, 9.

A6.

```
{'Ali': 75, 'Can': 60}
85 False False
Ali True
Can False
```

`75 in grades` tests the **keys**, so it is `False`.

A7.

```
1
not a number
divide by zero
3
done
```

`int("zero")` raises `ValueError`; `12 // 0` raises `ZeroDivisionError`; the loop goes on after each handler.

A8.

```
Counter at 16
15
```

`c1`: $0 \to 1$ (default `amount=1`) $\to 1 + 15 = 16$. `c2`: $10 \to 15$.

Part B

B1 **b** (`/` always gives a `float`, `1.0`) · B2 **a** (10 is the stop value, excluded) · B3 **b** (`[1, 2, [3, 4]]`) · B4 **b** (`in` tests keys) · B5 **a** (missing `:`) · B6 **b** (`int()` does not accept a decimal point) · B7 **a** (tuples are immutable) · B8 **c** (`__str__` returned `None`, not a string) · B9 **b** (a class-level list is created once and shared).

Part C

C1. `tag[5:9]` (or `tag[5:]`, `tag[-4:]`), `tag[-6]`, `t * 10`, `t > 15` (also `t >= 19`).

C2. `{}`, `in`, `+=`, `1`.

C3. `__init__`, `False`, `self`, `return`.

C4. (a) `ValueError` · (b) the text `'five'` · (c) line 2, in `to_volts` · (d) line 7, in `average` · (e) runtime · (f) line 11 · (g) the first call on line 10 finished normally: $(4.9 + 5.1) / 2 = 5.0$. In the exam the traceback is shorter (two functions), so (d) is already the main-program line. Read a traceback from the **last line up**: type and value, then the failing line, then each caller.

Part D

D1.

```
def classify_temp(t):
    if t < 0:
        return "Freezing"
    elif t < 15:
        return "Cold"
    elif t <= 25:
        return "Mild"
    else:
        return "Hot"
```

Results for `-1, 0, 14, 15, 25, 26`: Freezing, Cold, Cold, Mild, Mild, Hot.

D2.

```
def even_stats(values):
    count = 0
    total = 0
    for v in values:
        if v % 2 == 0:
            count += 1
            total += v
    return count, total
```

D3.

```
def count_letters(word):
    counts = {}
    for ch in word:
        if ch in counts:
            counts[ch] += 1
        else:
            counts[ch] = 1
    return counts
```

D4.

```
def parse_numbers(texts):
    numbers = []
    bad = 0
    for text in texts:
        try:
            numbers.append(float(text))
        except ValueError:
            bad += 1
    return numbers, bad
```

`float("")` also raises `ValueError`, so the empty string counts as bad.

D5.

```
class Thermostat:
    def __init__(self, name, min_temp, max_temp):
        self.name = name
        self.min_temp = min_temp
        self.max_temp = max_temp
        self.target = min_temp

    def set_target(self, t):
        if t < self.min_temp or t > self.max_temp:
            raise ValueError("target out of range")
        self.target = t

    def __str__(self):
        return f"{self.name}: target {self.target} ({self.min_temp}-{self.max_temp})"

th = Thermostat("TH1", 16, 28)
th.set_target(21)
print(th)                   # TH1: target 21 (16-28)
try:
    th.set_target(30)
except ValueError:
    print("rejected")
```

D6.

```
def in_range(values, low, high):
    result = []
    for v in values:
        if low <= v <= high:
            result.append(v)
    return result
```

A comprehension, `[v for v in values if low <= v <= high]`, gives the same list; the question asked for the loop form.

D7.

```
def largest(values):
    best = values[0]
    best_i = 0
    for i in range(1, len(values)):
        if values[i] > best:
            best = values[i]
            best_i = i
    return best, best_i
```

Start from the first element, not from `0`: a list of negative numbers would otherwise give `0`. Strict `>` keeps the first occurrence.

D8.

```
def order_total(prices, items):
    total = 0
    unknown = 0
    for item in items:
        if item in prices:
            total += prices[item]
        else:
            unknown += 1
    return total, unknown
```

D9.

```
class Inventory:
    def __init__(self):
        self.stock = {}

    def add(self, part, n):
        if n <= 0:
            raise ValueError("quantity must be positive")
        if part in self.stock:
            self.stock[part] += n
        else:
            self.stock[part] = n

    def take(self, part, n):
        if self.quantity(part) < n:
            raise ValueError("not enough " + part)
        self.stock[part] -= n

    def quantity(self, part):
        if part in self.stock:
            return self.stock[part]
        return 0

    def __str__(self):
        return f"Inventory: {len(self.stock)} part types"
```

`take` uses `quantity`, so a part that was never added counts as 0 and raises `ValueError` instead of `KeyError`.